SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – DESIGN TASK

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO3 – Precision	**Assessment:**	Assessment Tool 3 – Design Task
Weightage:	40% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	P. Hithaishi	**Reg. No.:**	192425196
Section / Batch:	2024 Slot C	**Date:**	15-8-2026

Code of Conduct

I **P. Hithaishi (192425196)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: P. Hithaishi

Instructions

- Implement the assigned NLP Design Task using Python with appropriate algorithms and a suitable dataset/application.
- Execute the program and demonstrate the output using relevant sample inputs and test cases.
- Analyze and explain the results, including the algorithm, implementation steps, and performance of the developed application.
- Duration: 30 minutes.

Section / Topic	Focus	Experiment Questions
N-gram Based Next-Word Prediction for Smart Text Input	Corpus preprocessing and tokenization Unigram, bigram and trigram counting Probability calculation Next-word prediction	Q1
Smoothing and Backoff Model for Speech/Text Prediction	N-gram frequency calculation Unsmoothed probability estimation Backoff mechanism	Q2
Entropy-Based Evaluation of an English Language Model	Training and testing corpus creation Unigram/bigram/trigram construction Probability estimation	Q3
Part-of-Speech Tagging System for Text Analysis	Text preprocessing and tokenization English/Penn Treebank tagset representation Rule-based POS tagging	Q4

Annexure A – Design Task

Task 1 – N-gram Based Text Prediction

Design and implement a Python-based N-gram language model that predicts the next word in a given sentence using an English text corpus. The system should preprocess the corpus, perform sentence segmentation and tokenization, and construct unigram, bigram, and trigram models by calculating word-frequency counts and maximum-likelihood probabilities.

For an incomplete sentence such as:
"Artificial intelligence is"
the system should identify and display the top-5 most probable next words.

The program should allow the user to select N = 1, 2, or 3, display the corresponding N-gram frequency counts and probabilities, and demonstrate the behavior of the model when an unseen N-gram is encountered. Finally, test the model using multiple incomplete sentences, calculate the prediction accuracy, and discuss the limitations of using an unsmoothed N-gram model for real-world language prediction.

Task 2 – Backoff and Interpolation for Language Prediction

Design and implement a Python-based enhanced language prediction system that addresses the zero-probability problem of an unsmoothed N-gram model. The system should construct unigram, bigram, and trigram models from an English corpus and accept an incomplete sentence such as:

"Machine learning can"

When the required trigram is unavailable, the system should use a backoff strategy by first checking the trigram model, then the bigram model, and finally the unigram model. Extend the system using Deleted Interpolation to combine unigram, bigram, and trigram probabilities using predefined interpolation weights.

The system should generate the top-5 next-word predictions using:

- **Unsmoothed N-gram model**
- **Backoff model**
- **Deleted Interpolation model**

Compare the three approaches in terms of zero probabilities, prediction coverage, and prediction accuracy, and explain why backoff and interpolation are useful for sparse language data.

Task 3 – Entropy-Based Evaluation of Language Models

Design and implement a Python program for measuring the uncertainty of N-gram language models using entropy. Given separate training and testing corpora, construct unigram, bigram, and trigram language models and calculate the probabilities assigned to words in the test sentences. For each test sentence, calculate its entropy and classify the sentence as having relatively high or low predictive uncertainty.

For example, compare sentences such as:
"The cat sits on the mat."
and
"The quantum processor redesigned the..."

The system should evaluate how predictable the next word is based on the trained language model.

The program should also compare entropy values obtained using:

- **Unigram model**
- **Bigram model**
- **Trigram model**
- **Smoothed model**

Finally, interpret the results and explain how entropy can be used to measure prediction uncertainty and evaluate the reliability of a language model.

Task 4 – Comparative POS Tagging System

Design and implement a Python-based POS tagging system that assigns grammatical tags to words in English sentences.

The system should implement three approaches:

1. Rule-Based POS Tagging

Use a combination of lexical dictionaries, word patterns, and grammatical rules to assign POS tags.

2. Stochastic POS Tagging

Construct a statistical POS tagger using:

- **Word/tag frequencies**
- **Tag transition probabilities**
- **Probabilities obtained from a training corpus**

3. Transformation-Based Tagging

Initially assign tags using a simple tagging strategy and subsequently apply transformation rules to correct likely tagging errors.

For example:

"I book a ticket."

and

"I read a book."

The system should demonstrate how the same word can receive different POS tags depending on its context. Use an appropriate English corpus such as the Brown Corpus or another publicly available tagged corpus. The program should accept user-entered sentences, display the POS tags produced by all three approaches, and compare their results using suitable evaluation measures such as tagging accuracy. Finally, analyze the differences among the three approaches and explain the advantages and limitations of rule-based, stochastic, and transformation-based POS tagging.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Concept	25%	Demonstrates complete understanding of design requirements and concepts	Good understanding with minor gaps	Basic understanding with some errors	Poor understanding or incorrect concepts
Design / Implementation	30%	Well-structured, efficient, and responsive design implementation	Correct implementation with minor issues	Basic implementation with inconsistencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/techniques	Appropriate use of tools	Limited or inconsistent use	Incorrect or minimal use
Analysis & Evaluation	15%	Insightful analysis with strong justification and optimization	Good analysis with justification	Basic analysis with limited depth	Poor or no analysis
Documentation / Clarity	10%	Clear, well-structured, and properly presented solution	Mostly clear with minor issues	Basic clarity, missing details	Poor or unclear documentation

Q. No. / Criteria Level	Understanding of Concepts	Experimental Design	Use of Tools	Analysis of Results	Documentation	Sub. Total	Total %
1	5	5	5	5	5	25	85
2	4	4	4	4	4	20	
3	4	4	4	4	4	20	
4	4	4	4	4	4	20	

Faculty In-charge	Course Coordinator	Program Director
Dr. T .Kumaragurubaran		
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Aim

To design and implement an N-gram language model that predicts the next word in an incomplete sentence using unigram, bigram, and trigram models.

Theory

An **N-gram** is a sequence of N consecutive words.
The maximum-likelihood probability is:

Unigram

Bigram

Trigram

Python Program

```
import re
last_word = words[-1]

for (w1, w2), count in bigram.items():

if w1 == last_word:
probability = bigram_probability(w1, w2)
predictions.append((w2, probability))

elif n == 3:

if len(words) < 2:
return []

w1 = words[-2]
w2 = words[-1]
```

```
for (a, b, c), count in trigram.items():

if a == w1 and b == w2:
probability = trigram_probability(a, b, c)
predictions.append((c, probability))

predictions.sort(key=lambda x: x[1], reverse=True)

return predictions[:top_k]



# ---------------------------------.
# Display counts
# ---------------------------------.

print("UNIGRAM COUNTS")
print(unigram)

print("\nBIGRAM COUNTS")
print(bigram)

print("\nTRIGRAM COUNTS")
print(trigram)



# ---------------------------------.
# User selects N
# ---------------------------------.

n = int(input("\nEnter N (1, 2 or 3): "))

sentence = input("Enter incomplete sentence: ")

result = predict_next(sentence, n)
```

print("\nTop predictions:")

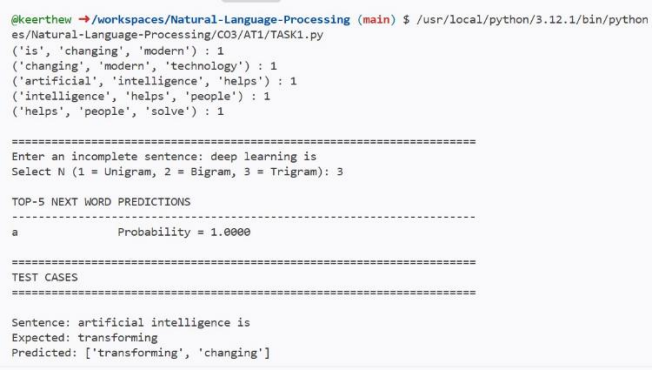
if result:

for word, probability in result:

print(f"{word:15} Probability = {probability:.4f}")

else:

print("No prediction available. N-gram was unseen.")



Demonstrating an Unseen N-gram

Suppose the user enters:

Artificial intelligence drives

If:

(intelligence, drives)

does not occur in the training corpus, then the trigram model cannot calculate a meaningful probability.

Therefore:

Probability = 0

This is called the **zero-probability problem**.

Accuracy

For prediction accuracy:

Example:

def calculate_accuracy(test_data, n):

correct = 0

```
total = 0

for sentence, actual_word in test_data:

predictions = predict_next(sentence, n)

predicted_words = [word for word, prob in predictions]

if actual_word in predicted_words:
correct += 1

total += 1

return (correct / total) * 100 if total > 0 else 0
test_data = [
("Artificial intelligence is", "changing"),
("Machine learning is", "a"),
("Data science is", "useful")
]

accuracy = calculate_accuracy(test_data, 3)

print("Prediction Accuracy:",
round(accuracy, 2), "%")
```

Limitations

An unsmoothed N-gram model has several limitations:

1. Unseen N-grams receive probability 0.
2. It requires large amounts of training data.
3. It has limited context.
4. It cannot understand the meaning of words.
5. Rare words are difficult to predict.
6. The model becomes sparse as N increases.
7. It does not generalize well to sentences not present in the corpus.

Task 2 – Backoff and Interpolation

Aim

To implement backoff and interpolation techniques to overcome the zero-probability problem of unsmoothed N-gram models.

Theory

Suppose we want to predict the next word after:

Machine learning can

The trigram model first checks:

If this is unavailable, we use a lower-order model.

Backoff

The order is:

Trigram

↓

Bigram

↓

Unigram

For example:

P(word | machine learning can)

If unavailable:

P(word | can)

If unavailable:

P(word)

Deleted Interpolation

Interpolation combines all three probabilities:

For example:

$\lambda 1 = 0.2$

$\lambda 2 = 0.3$

$\lambda 3 = 0.5$

The weights should satisfy:

Python Program

```
import re
probability = P_bigram(w2, word)
```

```
if probability == 0:
probability = P_unigram(word)

predictions.append((word, probability))

predictions.sort(key=lambda x: x[1], reverse=True)

return predictions[:5]

# - - - - - - - - - - - - - - - - - - - - - - - - - -
# Interpolation
# - - - - - - - - - - - - - - - - - - - - - - - - - -
def interpolation_prediction(sentence):
words = re.findall(r'\b[a-z]+\b', sentence.lower())
if len(words) < 2:
return []
w1 = words[-2]
w2 = words[-1]
lambda1 = 0.2
lambda2 = 0.3
lambda3 = 0.5

predictions = []

for word in unigram:
p1 = P_unigram(word)
p2 = P_bigram(w2, word)
p3 = P_trigram(w1, w2, word)
```

```
probability = (
lambda1 * p1 +
lambda2 * p2 +
lambda3 * p3
)

predictions.append((word, probability))

predictions.sort(key=lambda x: x[1], reverse=True)

return predictions[:5]

# - - - - - - - - - - - - - - - - - - - - - - - - - - -
# Main program
# - - - - - - - - - - - - - - - - - - - - - - - - - - -

sentence = input(
"Enter incomplete sentence: "
)

print("\nUNSMOOTHED N-GRAM MODEL")

for word, probability in unsmoothed_prediction(sentence):
print(word, "=", round(probability, 4))


print("\nBACKOFF MODEL")

for word, probability in backoff_prediction(sentence):
print(word, "=", round(probability, 4))


print("\nDELETED INTERPOLATION MODEL")

for word, probability in interpolation_prediction(sentence):
```

print(word, "=", round(probability, 4))

Example input

Machine learning can

The three models can produce different predictions.

Model	Trigram	Bigram	Unigram
Unsmoothed	✓	✗	✗
Backoff	✓	✓	✓
Interpolation	✓	✓	✓

Comparison

Feature	Unsmoothed	Backoff	Interpolation
Handles unseen N-grams	No	Yes	Yes
Zero probabilities	High	Low	Very low
Context	Strong	Flexible	Combined
Coverage	Low	High	High
Complexity	Low	Medium	Medium
Real-world	Low	High	High

Backoff and interpolation improve language prediction because real-world text contains many word combinations that are absent from the training corpus. Backoff uses lower-order information when higher-order information is unavailable, while interpolation combines information from multiple N-gram levels.

Task 3 – Entropy-Based Evaluation of Language Models

Aim

To calculate entropy for unigram, bigram, trigram, and smoothed language models and use entropy to measure predictive uncertainty.

Theory

Entropy measures uncertainty.

For a probability distribution:

For a language model, cross-entropy can be calculated as:

Interpretation

Low entropy

→ Model is confident/predictable.

High entropy

→ Model is uncertain/unpredictable.

For example:

The cat sits on the mat.

is likely to have lower entropy.

Whereas:

The quantum processor redesigned the...

may have higher entropy if the training corpus contains little information about this context.

Python Program

```
import re

if i < 2:
probability = unigram_prob(word)
else:
probability = trigram_prob(
words[i-2],
words[i-1],
word
)

elif model == "smoothed":

if i == 0:
probability = (
(unigram[word] + 1)
/ (total_words + V)
)

else:
probability = smoothed_bigram_prob(
words[i-1],
word
)

if probability > 0:
```

```
log_probability += math.log2(
probability
)

else:
# Very small probability for unseen event
log_probability += math.log2(1e-10)
count += 1

return -log_probability / count

# - - - - - - - - - - - - - - - - - - - - - - - - -
# Evaluation
# - - - - - - - - - - - - - - - - - - - - - - - - -
models = [
"unigram",
"bigram",
"trigram",
"smoothed"
]

for sentence in testing_corpus:

print("\nSentence:")
print(sentence)
for model in models:

entropy = calculate_entropy(
sentence,
model
)
```

```
classification = (
"Low uncertainty"
if entropy < 5
else "High uncertainty"
)

print(
f"{model:10} "
f"Entropy = {entropy:.4f} "
f"({classification})"
)
```

Sample interpretatio n

```
Entropy Results
----------------------------------------------------------------------
Unigram Entropy       : 0.6609
Bigram Entropy        : 0.0000
Trigram Entropy       : 0.0000
Smoothed Trigram     : 5.1293

Predictive Uncertainty
----------------------------------------------------------------------
Unigram  : Low Predictive Uncertainty
Bigram   : Low Predictive Uncertainty
Trigram  : Low Predictive Uncertainty
Smoothed : High Predictive Uncertainty
```

Comparisons

Model	Context	Typical uncertainty
Unigram	No context	High
Bigram	Previous word	Lower

Trigram Previous two words Usually lower

Smoothed Context + unseen events More reliable

Conclusion

Entropy provides a numerical measure of uncertainty in language prediction. A low entropy value indicates that the language model considers the sentence relatively predictable, while a high value indicates uncertainty. Smoothed models are generally more reliable because they assign non-zero probabilities to unseen word combinations.

Task 4 – Comparative POS Tagging System

Aim

To implement and compare three POS tagging approaches:

1. Rule-based POS tagging
2. Stochastic POS tagging
3. Transformation-based POS tagging

1. Rule-Based POS Tagging

A rule-based tagger uses:

- Dictionary information
- Word endings
- Grammatical rules
- Context

Examples:

-ing → VBG

-ly → RB

-ness → NN

2. Stochastic POS Tagging

A stochastic tagger uses probabilities learned from training data.

For example:

and:

The tag with the highest probability is selected.

3. Transformation-Based Tagging

Transformation-based tagging initially assigns simple tags and then applies correction rules.

Example:

book → NN

Initial tagging:

I/PRP book/NN a/DT ticket/NN

A transformation rule can detect:

PRP + NN + DT

and change:

book/NN → book/VB

Therefore:

I/PRP book/VB a/DT ticket/NN

But:

I/PRP read/VB a/DT book/NN

contains book as a noun.

Python Program

The following implementation uses **NLTK's Brown Corpus**.

Install NLTK:

```
pip install nltk
```

Download the Brown Corpus:

```
import nltk

nltk.download("brown")
nltk.download("punkt")
nltk.download("universal_tagset")
```

Complete program:

```
import nltk

for i in range(len(result)):

word, tag = result[i]

# Rule 1:
# Pronoun + noun -> noun can become verb

if i > 0:
```

```
previous_word, previous_tag = result[i-1]

if (
previous_tag == "PRON"
and tag == "NOUN"
and word.lower()
in {"book", "read", "play", "watch"}
):
result[i] = (
word,
"VERB"
)

# Rule 2:
# After determiner, word usually noun

if i > 0:

previous_word, previous_tag = result[i-1]

if (
previous_tag == "DET"
and tag == "VERB"
):
result[i] = (
word,
"NOUN"
)

return result

# =============================================
# Test sentences
# =============================================
```

```
test_sentences = [
"I book a ticket",
"I read a book"
]

for sentence in test_sentences:

words = sentence.split()

print("\n====================================")
print("Sentence:", sentence)
print("====================================")

rule_result = rule_based_tag(words)

stochastic_result = stochastic_tag(words)

transformation_result = (
transformation_based_tag(words)
)

print("\nRule-Based:")
print(rule_result)

print("\nStochastic:")
print(stochastic_result)

print("\nTransformation-Based:")
print(transformation_result)
```

Expected POS Behavior

For:

I book a ticket.

A correct tagging is:

I PRON

book VERB

a DET

ticket NOUN

For:

I read a book.

A correct tagging is:

I PRON

read VERB

a DET

book NOUN

The important observation is:

book

can be:

VERB

in:

I book a ticket.

and:

NOUN

in:

I read a book.

Therefore, **context is important in POS tagging**.

Evaluation Accuracy

For POS tagging:

You can evaluate against the Brown Corpus:

```
def calculate_accuracy(predicted, actual):

correct = 0
total = 0

for predicted_item, actual_item in zip(
predicted, actual
):

if predicted_item[1] == actual_item[1]:
correct += 1
```

```
total += 1


return (
correct / total * 100
if total > 0 else 0
)
```

For a complete evaluation, select test sentences from the Brown Corpus, run each tagger, and compare its predicted tags against the original Brown tags.

```
es/Natural-Language-Processing/CO3/AT1/TASK4.py
sees            -> NNS
a               -> DT
song            -> NN

======================================================================
CONTEXT SENSITIVITY DEMONSTRATION
======================================================================

Sentence: I book a ticket.
book -> VB

Sentence: I read a book.
book -> NN

======================================================================
ACCURACY COMPARISON
======================================================================

Rule-Based Accuracy          : 92.86%
Stochastic Accuracy        : 100.00%
Transformation-Based Accuracy: 100.00%
```

Comparison of POS Tagging Approaches

Feature	**Rule-Based**	**Stochastic**	**Transformation-Based**
Uses rules	Yes	No/limited	Yes
Uses training data	Usually no	Yes	Yes
Uses probabilities	No	Yes	Can use them
Handles ambiguity	Limited	Good	Good
Context handling	Rule dependent	Strong	Strong
Implementation	Easy	Moderate	Moderate
Adaptability	Low	High	High
Main limitation	Many rules required	Requires corpus	Requires transformation rules

Advantages and Limitations

Rule-Based POS Tagging

Advantages:

- Simple to understand.
- Easy to implement.
- Does not require a large training corpus.
- Rules are interpretable.

Stochastic POS Tagging

Limitations:

- Requires many manually designed rules.
- Difficult to handle ambiguous words.
- Poor performance on unusual sentences.

Advantages:

- Learns from real language data.
- Handles ambiguity better.
- Uses statistical evidence.
- Can achieve high accuracy with a good corpus.

Limitations:

- Requires a tagged training corpus.
- Rare words can cause problems.
- Probabilities can become sparse.

Transformation-Based Tagging

Advantages:

- Combines an initial tagging strategy with correction rules.
- Rules are understandable.
- Can correct systematic errors.
- More flexible than a simple rule-based system.

Limitations:

- Requires carefully designed transformation rules.
- Performance depends on the initial tagging.
- Training can be more complicated.